

Software Architecture Specification (SAS)

Neighbourhood WatchDog

version 4.1 Updated 29 September 2026

1. Introduction

This Software Architecture Specification (SAS) document describes the architectural design of the Neighbourhood WatchDog system. It outlines the system's major components, their responsibilities, interactions, and the technologies used to implement them. The document serves as a reference for developers, stakeholders, and maintainers by providing a comprehensive overview of the software architecture and the rationale behind key design decisions.

The architecture of Neighbourhood WatchDog is organised into six primary subsystems: Video Ingestion, AI Detection Engine, Alert and Event Management, User and Access Control, Monitoring Dashboard, and Data and Storage Management. Together, these subsystems provide a modular and scalable foundation capable of supporting real-time video processing, secure user management, and reliable data persistence. Technologies such as MediaMTX, FFmpeg, YOLOv8, DeepSORT, FastAPI, WebSockets, Next.js, PostgreSQL, AWS Cognito, and Amazon S3 are utilised to ensure the system remains performant, extensible, and suitable for deployment in community-based security environments.

1.1 Purpose

The purpose of this document is to communicate the architectural structure of the system, define the relationships between its components, and establish a shared understanding of the design principles that guide its implementation and future evolution.

2. Architectural Requirements

2.1 Architectural patterns

Pattern	Application in WatchDog	Purpose	Rationale
Client-server	Next.js browser client consumes FastAPI REST/WebSocket services.	The Next.js frontend communicates with backend APIs and WebSocket services through a client-server model.	Separates interaction/UI from domain and persistence concerns.
Event-driven	Detection/annotation posts result in backend WebSocket broadcast to browser clients.	Real-time stream processing enables asynchronous communication between video ingestion, AI detection services, and alert management.	Supports live operational updates without refresh polling.

2.2 Design Patterns

Design Pattern	Implementation
Strategy	The notification service uses a strategy to send notifications out through different channels (Email, Push notifications or Whatsapp) and allows one to add channels easily.
Observer	When an AI worker posts a detection or annotation to the FastAPI backend, the backend broadcasts alert and annotation updates through WebSockets. Dashboard clients subscribed through WebSocket connections receive updates immediately and refresh alerts or bounding-box overlays without polling.
State	The camera playback UI moves between distinct states: connecting, live, and unavailable/offline. When a user selects a camera, the frontend initiates WHEP/WebRTC playback and displays the appropriate state based on connection success, stream availability, or failure. Closing the camera view ends the playback session and returns the

Design Pattern	Implementation
	component to its initial state. Moreover, the Edge Agent also moves between distinct states: setup, running and off.
Factory	A factory is used when a user wants to retrieve a policy for a particular type of notification. The user passes the event type into the factory and is given the correct notification policy object and it can be used to send the notification to the user through the appropriate channels for that policy
Chain of Responsibility	Requests pass through sequential validation and authorisation stages: authentication, role/property access checks, request validation, internal-agent token validation, and finally route/service execution. FastAPI middleware and dependency functions naturally support this pattern.
Command	Camera actions such as enable, disable, start publisher, stop publisher, start detection, and stop detection, can be represented as commands. This would make operations easier to queue, retry, log, audit, and potentially execute remotely through the WatchDog Agent.
Builder	The builder design pattern is used to create the different policies for notifications. For example, there is a policy for joining a neighbourhood and a user can select which channels should be used to send the notification through (Email, Push notifications or Whatsapp). This allows for flexible adding or removing channels from a policy without have to rewrite large sections of code.

2.3 Architectural Constraints

- Existing CCTV and IP cameras must provide RTSP streams.
- Processing multiple video streams simultaneously can be computationally expensive and doing this in the cloud can incur a great monetary cost, further, doing it locally may be slow due to hardware limitations of the user's machine.
- Real-time video processing must minimize latency while supporting multiple concurrent streams.
- Sensitive information must remain encrypted during transmission.
- Secrets and credentials may not be committed to source control.

- All deployments must be reproducible using Infrastructure as Code and containerization technologies.
- The system must support independent scaling of AI processing services.
- Production deployments must remain accessible through public URLs.
- Services must support horizontal scalability where appropriate.

2.4 Architectural Diagram

Architecture Diagramv2

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision
Availability: $\geq 99.5\%$ uptime	Made use of a load balancer as well as multi-AZ database deployment
Performance: Support for 500+ concurrent users	Deployed with ECS, AWS' containerised management service deployed with automatic scaling policies
Data encryption for sensitive information	Use of AWS RDS and AWS Secrets Manager both of which encrypt data at rest using AES-256 and TLS 1.3 for secure communication between services. Moreover, MFA is used through AWS Cognito
Reliability: Recovery within 5 minutes after failure	Automated backups, failover replication, and monitoring with automated recovery scripts.
Scalability: support for up to 200% workload increase without major architectural changes	ECS with an Application Load Balancer allows horizontal scaling by increasing the service's desired task count with no architecture changes. The underlying EC2 ASG (auto scaling group) scales out to provide for additional tasks
Maintainability: new features/fixes deployable within 2 hours	CI/CD via GitHub Actions automates images build, ECR push and ECS service update on every push to the main branch of the repository. Deployments complete within a matter of minutes

3. Technology Requirements

3.1 Frontend Technologies

Technology	Role	Justification
Next.js, React, TypeScript	Web Dashboard	Typed component-based UI with route structure suitable for public/private flows.
Tailwind CSS	Styling	Supports consistent styles and responsive design.
WebSockets	Real-Time Communication	Supports live alert updates

3.2 Backend Technologies

Technology	Role	Justification
FastAPI	REST/WebSocket API and domain logic	High-performance asynchronous APIs.
Python	Backend Development	Extensive AI and computer vision ecosystem.
Node.js	Supporting Services	Efficient event-driven processing.
SQLAlchemy	ORM	Simplifies database interactions.

3.3 AI Technologies

Technology	Role	Justification
YOLOv8	Object Detection	Provides baseline visual detection/tracking at edge.
DeepSort	Object Tracking	Person detection and within-stream tracking.
OpenCV	Image Processing	Provides a robust, high-performance foundation for handling image and video data before and during model inference.
PyTorch	Machine Learning Framework	PyTorch is an ideal framework for building and deploying AI detection systems due to its flexibility, dynamic computation,

Technology	Role	Justification
		and rich ecosystem of pre-trained vision and NLP models.

3.4 Infrastructure Technologies

Technology	Role	Justification
Docker Compose, Caddy	Reproducible container runtime and API TLS/reverse proxy	Supports local and EC2 runtime topology.
GitHub Actions, Vercel	CI and frontend deployment	Automates validation and public frontend deployment.
MediaMTX	Stream relay and WHEP/WebRTC egress	Separates source camera from consumers; provides low-latency browser protocol.
FFmpeg	Camera source conversion and relay publication	Mature protocol support for heterogeneous RTSP inputs.
Redis	Caching and Background Processing	Delivers the ultra-low latency, high-throughput messaging, and real-time state management required to process live video frames and trigger instant alerts.
Amazon S3	Video Clip Storage	Provides cheap, reliable, and scalable storage that easily connects to AI and video processing tools.
AWS Cognito	Authentication and Authorization	Handles user registration, secure login, password resets, and multi-factor authentication (MFA).

3.5 Technology Selection Rationale

The selected technologies collectively support:

- High availability
- Horizontal scalability
- Real-time event processing
- Secure authentication mechanisms
- Efficient video stream processing
- Automated deployments
- Maintainable and modular development practices

4. API Contracts

Neighbourhood WatchDog exposes five contract surfaces: the browser REST API, the paired WatchDog Agent control API, real-time WebSockets, MediaMTX WHEP playback, and the MediaMTX publish-authorisation callback. The FastAPI backend owns the REST, WebSocket and callback contracts; the dashboard and native Agent consume them.

The OpenAPI standard service contract for the backend is hosted at this site <https://api.neighbourhoodwatchdog.co.za/docs>

5.1 Common HTTP conventions

All REST requests and responses use application/json unless a media-specific contract says otherwise.

- **Browser authentication:** protected browser routes resolve the caller from the Cognito/JWT-backed user context. Controller-level role checks enforce resident, property-administrator, neighbourhood-administrator and system-administrator permissions.
- **Agent authentication:** Agent runtime endpoints use a paired edge-Agent credential; detection ingestion currently requires X-Internal-Token.
- **Identifiers:** resource identifiers are UUIDs unless stated otherwise.
- **Success responses:** current endpoints are not yet fully normalised. Some return { "status", "message", "data" }, some return a schema object, and deletion endpoints return 204 No Content.
- **Error responses:** the current FastAPI implementation returns { "detail": "..." } for many failures and the standard FastAPI 422 validation format. A unified public error envelope is **Planned**.

Recommended future error envelope:

```
{
  "error": "forbidden",
  "cause": "The current user is not allowed to edit this camera.",
  "suggestion": "Ask a property or neighbourhood administrator to
perform this action.",
  "request_id": "correlation-id"
}
```

5.2 Browser REST API

The following routes cover the principal Demo 2 browser flows. Request and response fields are defined by the Pydantic schemas in `backend/app/schemas/`, which remain the field-level source of truth.

Method	Path	Auth / role	Purpose
GET	/health	No	Returns { "status": "ok" } for deployment smoke checks.
POST	/auth/signup	No	Register a user from the sign-up request schema. Rate limited to 3/minute.
POST	/auth/confirm	No	Confirm account registration with a confirmation code.
POST	/auth/login	No	Authenticate a registered user. Rate limited to 5/minute.
POST	/auth/resend-code	No	Re-send an account-confirmation code.
GET	/auth/me	User	Return the current authenticated user's profile.
POST	/neighbourhood/create-neighbourhood	Resident	Create a neighbourhood using name, location and initial property context.
POST	/neighbourhood/join	User	Submit a join request using a neighbourhood join code.
GET	/neighbourhood/join-requests	User	List join requests visible in the caller's administrative context.
PATCH	/neighbourhood/join-requests/{request_id}	Neighbourhood administrator	Approve or deny a join request using { "action": "..." }.
POST	/properties/create-property	Resident	Create a property using its address and type.
GET	/properties/my-properties	Resident	List the caller's properties.
GET	/properties/{property_id}	Resident	Fetch property details, including related users, neighbourhood and cameras.
POST	/camera/register-camera	Resident	Register a camera and associate it with the caller's property.
GET	/camera/property/{property_id}	Resident	List cameras belonging to a property.
PATCH	/camera/{camera_id}	Resident	Update camera details, including its enabled state.
DELETE	/camera/{camera_id}	Resident	Permanently deregister a camera. Returns 204.

Method	Path	Auth / role	Purpose
GET	/cameras/{camera_id}/settings	Neighbourhood/property/system administrator	Retrieve the confidence threshold and detection zones.
PATCH	/cameras/{camera_id}/settings	Neighbourhood/property/system administrator	Update { "confidence_threshold": number }.
POST	/cameras/{camera_id}/zones	Neighbourhood/property/system administrator	Create a named polygon detection zone.
DELETE	/cameras/{camera_id}/zones/{zone_id}	Neighbourhood/property/system administrator	Remove a detection zone. Returns 204.
GET	/alerts/{neighbourhood_id}	Authorised user	List alerts using optional status, camera, detection type, date and pagination filters.
PATCH	/alerts/{alert_id}/acknowledge	Authorised user	Acknowledge an alert and return its updated state.
GET	/audit/get-audit-logs	System administrator	Retrieve paginated and filterable audit records.

Browser safety rule: a browser-facing response must not disclose a source-camera RTSP URL, Agent API key, internal token or MediaMTX publisher password.

5.3 Paired WatchDog Agent control API

The native Agent calls this surface during first-time pairing and when reconciling the active camera runtime. Its credential is retained in the operating-system keyring, not in browser-accessible storage.

Method	Path	Caller / authentication	Purpose	Realises
GET	/pairing-token/{property_id}	Resident	Issue a pairing token for the selected property. The pairing service is responsible for expiry and one-time validation.	R1.1.4, R7.4.1.3
GET			Exchange a valid pairing token for a	

Method	Path	Caller / authentication	Purpose	Realises
	/pairing-token/ token/ {pairing_token}	Native Agent during setup	property-scoped Agent credential.	R1.1.4, R4.1.6
GET	/internal/ cameras/enabled	Paired Agent credential	Return only enabled cameras for the Agent's property: camera ID, Agent-only RTSP URL, neighbourhood ID, threshold and per- camera MediaMTX publisher credentials.	R1.1.3- R1.4.2, R2.1.4
POST	/internal/ detections	X-Internal-Token	Persist a validated detection result; a qualifying event can create an alert. Returns 201. Must be network- restricted in production.	R2.1.3- R2.2.3, R3.1
POST	/api/stream/ cameras/ {camera_id}/ annotations	Paired Agent credential	Broadcast current annotation/track data to connected dashboard clients. Returns { "status": "broadcasted" }.	R2.1.4- R2.1.5, R5.3

5.4 Real-time WebSocket contracts

The current implementation provides two server-to-browser JSON channels. Both emit { "event": "ping" } after 30 seconds without a received client frame to preserve connection health.

Camera annotation channel - /api/stream/cameras/{camera_id}/annotations/ws

```
{
  "camera_id": "camera-uuid",
  "event": "annotation",
  "tracks": [
    {
      "track_id": 17,
      "bbox": [84, 26, 201, 317],
      "detection_type": "person",
```

```

        "confidence": 0.91
    }
]
}

```

Direction	Event / payload	Cadence	Realises
Server → Browser	{ "camera_id", "event": "annotation", "tracks": [...] } }	Whenever the Agent posts an annotation payload	R2.1, R2.4.1, R5.3
Server → Browser	{ "event": "ping" } }	30 seconds of idle receive time	Connection health
Browser → Server	Any text frame / heartbeat	Keeps the current session alive; no browser command message is defined.	Connection health

The browser renders a detection overlay using bbox, detection_type, confidence and optional tracking data. This Agent-originated payload should be promoted to a Pydantic schema before production.

Neighbourhood alert channel - /alerts/{neighbourhood_id}/ws?token={token}

```

{
    "event": "new_alert",
    "camera_id": "camera-uuid",
    "detection_type": "HUMAN_PRESENCE",
    "confidence": 0.91
}

```

Direction	Event / payload	Cadence	Realises
Server → Browser	{ "event": "new_alert", "camera_id", "detection_type", "confidence", ... } }	When an alert is broadcast to a neighbourhood	R3.1, R5.1.1
Server → Browser	{ "event": "ping" } }	30 seconds of idle receive time	Connection health
Browser → Server	Any text frame / heartbeat	No client command discriminator exists; acknowledgement is the REST	R5.1.3

Direction	Event / payload	Cadence	Realises
		PATCH /alerts/{alert_id}/ acknowledge call.	

Security status - In progress: the alert WebSocket accepts a token query parameter but does not yet validate it, and the annotation WebSocket has no visible user authorisation check. Before production, both handshakes must authenticate the user and enforce the same camera/neighbourhood access policy as the REST API.

5.5 Media relay and playback contracts

MediaMTX publish-authorisation callback - POST /internal/mediamtx/auth

MediaMTX calls the backend before accepting a publisher. It submits a JSON object containing user, password, action, path, protocol, ip, id and query.

Caller	Accepted operation	Contract	Result
MediaMTX	publish	Path must match cameras/<camera-uuid>. The camera must exist and be enabled. Username is camera-<camera-uuid> and password is the HMAC-derived per-camera password.	204 No Content when authorised; 401 for invalid camera/path/credential; 403 for unsupported action.
MediaMTX	read or playback	The current callback permits these actions directly. Browser-side stream authorisation remains an in-progress security requirement .	204 No Content

Browser WHEP/WebRTC playback - POST {NEXT_PUBLIC_MEDIAMTX_WEBRTC_URL}/cameras/{camera_id}/whep

Playback begins only after explicit camera selection. The dashboard creates a receive-only WebRTC offer and posts its SDP to MediaMTX:

POST /cameras/{camera_id}/whep
Content-Type: application/sdp

<browser WebRTC offer SDP>

MediaMTX returns an SDP answer and may include a Location header for the WHEP session resource. On camera-modal close, the dashboard closes the peer connection and sends DELETE to that session URL when supplied.

Direction	Contract	Dashboard state	Realises
Browser → MediaMTX	POST receive-only SDP offer	connecting	R1.2.4, R5.3
MediaMTX → Browser	SDP answer and WebRTC media tracks	live	R1.2.4, R5.3
Browser → MediaMTX	DELETE WHEP session on component unmount, when supplied	idle	R1.2.4
Failure	Non-2xx WHEP response or failed peer connection	unavailable	R5.2.1

5.6 Contract ownership and source of truth

Surface	Owner	Authoritative implementation source
Browser REST	FastAPI backend	backend/app/api/controllers/ and backend/app/s
Agent control	FastAPI backend and native Agent	internal_cameras.py, detection.py, stream.py, pairing_token.py and Agent runtime modules
Real-time WebSockets	FastAPI backend and dashboard	alert.py, stream.py and frontend WebSocket ho
WHEP/ WebRTC	MediaMTX and CameraFeed.tsx	MediaMTX WHEP contract and browser imple
Publish authorisation	FastAPI backend and MediaMTX	internal_cameras.py and MediaMTX configurati

Surface	Owner	Authoritative implementation source

6. Deployment

7.1 Deployment Requirements

Live, Accessible System

Available at: <https://neighbourhoodwatchdog.co.za>

Environment Parity

There are 3 distinct environments:

- Development: Local docker-compose, run manually locally.
- Staging: A single EC2 instance running the full stack via Docker compose deployed automatically by a GitHub Actions workflow triggered on pushes to the dev branch
- Production: a separate architecture on AWS:ECS on EC2 for the backend, a standalone EC2 instance for mediamtx (relay for sending the streaming to the client from the edge agent), and RDS for the database. Deployed automatically by a GitHub Actions workflow triggered on every push to the main branch.

Infrastructure as Code / Containerisation

Infrastructure as code is managed in **main.py**. All the services are containerised via Docker and deployment is driven by declarative task definitions (with AWS ECS) and Docker Compose files rather than manual console operations.

Secrets Management

No credentials are committed to the GitHub repository. Staging generates it .env file at deploy time from the Github Actions secrets. Production uses AWS Secrets Manager which holds all sensitive configuration which is then injected into the different ECS containers via the task definitions secrets field.

Rollback strategy

The system supports rollback in the following ways:

- 1. Application rollback (ECS task definitions): Each deploy registers a new immutable task definition rather changing an existing one. So previous versions are still available and can be rolled back to by pointing the service at the last known good revision.
- 2. Coniguration rollback (Secrets Manager): The Secrets Manager retains prior versions automatically thus allowing rollback.

Deployment Diagram

Prod Deployment Diagram

Staging Deployment Diagram

CI/CD Pipeline Diagram

CI/CD Pipeline Diagram

8. NFR Testing

NFR Traceability Matrix

NFR ID	Quantified requirement	Design tactic / implementation	Verification test / tool
QR-01	On the fixed labelled person-detection evaluation harness, human detection precision shall be at least 60% and recall shall be at least 60%.	The person detector uses a confidence threshold of 0.25 and NMS IoU threshold of 0.70, selected through a 49-candidate tuning search. DeepSORT tracking and TEMPORAL_CONFIRMATION_FRAMES=3 prevent alerts from being raised from a single unconfirmed frame.	ai/evaluation/run_baseline.py executed against the fixed 24-it labelled person-detection harness. Temporal confirmation is verified separately by ai/test/test_alert_confirmation

Availability

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
QR-02	ECS recovers killed task to health within 360s	ECS circuit breaker + ASG. Health check threshold is set at 5 x 30s to avoid premature failover on transient blips	Manually run aws ecs stop-task, time until ALB target group reports healthy again	<= health-check grace period (360s once reverted from the temporary 10s)	341s Log of number of services running
QR-03	mediamtx stream resumes within 60s of a mediamtx restart	Edge agent RTSP reconnect/retry with backoff	Restart mediamtx container, time until WebRTC stream is viewable again	< 60s	5.5s (T0 12:02:01.3Z, publishing stream at 12:02:09.19Z)

Security

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
QR-04	Zero high/critical dependancy CVEs on main	Automated dependancy scanning in CI	pip audit + pnpm audit	0 high or critical	0 known vulnerabilities found 0 findings
QR-05	Zero medium+ severity findings on staging	Input validation, security headers, limited exposure	OWASP ZAP baseline scan again staging	0 medium+	0 (There was a false positive critical)
QR-06	0 secrets committed to the repository	Making use of GitHub Actions secrets and Secrets Manager	gitleaks	0 findings	0 findings 0 findings

Recoverability

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
QR-07	A failed production deployment can be rolled back to the previous health task definition within 5 minutes	ECS task definition rollback and the deployment circuit breaker	Force a bad deploy and run the documented rollback command and test time until health	<= 5 mins	service never left healthy state because the circuit breaker prevented the bad revision from ever reaching majority healthy status. The broken task auto stopped within seconds. 2 out of 3 good tasks kept serving throughout.
QR-08	Edge agent continues operating in last-known camera config for at least indefinitely if the backend is not reachable	Local caching of the last successful request for the list of enabled cameras	Turn the backend off and on and observe whether the stream continues to be pushed on the list of existing cameras	runs indefinitely	runs indefinitely thanks to caching of camera configurations. And when it does send out requests, it sends them out with exponential backoff so it will not further break the backend if there are issues with it.

Scalability

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
QR-09	ECS launches an additional task within 3	ASG and ECS target-	Locust load test sustained past the	<= 3 minutes	+38s (alarm transitioned to ALARM at

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
	minutes of sustained CPU and/or memory threshold being exceeded under load	tracking auto-scaling policy	threshold, watch describe-services for scale out event		19:48:44Z UTC and the earliest observable capacity improvement in Locust data was at 19:49:22Z UTC)
QR-10	p95 latency stays under 3000ms at 500 concurrent virtual users	Connection pooling + indexing + auto-scaling and Redis caching for selected endpoints	Sustained Locust load test, 500 VUs, 10min	p95 < 3000ms at 500 Virtual Users	2700ms at 500 users 2700ms at 500 users
QR-11	Error rate at peak load	Connection pool limit	Locust	<1%	<u>0.20%</u>

Maintainability

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
QR-12	SonarQube Maintainability rating of A on code in main branch	Code quality requirements	SonarQube	A rating	A rating alt text

Accessibility

ID	Quantified Requirement	Tactic in SAS	Test / tool	Target	Actual
QR-13	Google Lighthouse accessibility score above 95 on production frontend	Code quality requirements	Google Lighthouse	95 accessibility rating	96 accessibility rating 96 accessibility rating